# Coding Challenge: Build a Small Enterprise AI Assistant

| | |
|---|---|
| **Timebox** | Maximum 8 hours |
| **AI-assisted development** | Explicitly encouraged |
| **Submission** | Source for a runnable application and a README |

Build a small AI assistant for first-level IT and HR questions on Cloudflare infrastructure.
We are looking for a functioning prototype with deliberate technical choices. Researching and using existing frameworks, libraries, starter templates and platform capabilities is encouraged. Keep the implementation focused and understandable.

## The scenario

Your application contains exactly three agents:

- **Orchestrator:** receives all user messages, delegates specialist work, asks follow-up questions and delivers the final response.
- **IT specialist:** handles delegated technical questions and basic troubleshooting.
- **HR specialist:** handles delegated questions about employee processes and company policies.

The orchestrator is the user-facing assistant. There is no separate interaction agent. Both specialists return their contributions only to the orchestrator.
Use fictional company information and simulated systems. Detailed IT or HR expertise is not required.

## 1. Agent orchestration

The orchestrator delegates work to the specialists and should:

- Delegate the user question to the right specialist.
- Run independent specialist tasks concurrently.
- Run dependent tasks sequentially and pass relevant results to the next specialist.
- Combine specialist contributions into one coherent response.
- Communicate missing information, unavailable capabilities and failed execution honestly.

Each agent has its own instructions, its own message history and its own set of capabilities. Keep execution bounded with suitable limits and timeouts. Separate deployments for each agent are not required.

## 2. Tools, MCP and skills

Implement three different mechanisms:

| Mechanism | Minimum requirement |
|---|---|
| Native tool | One callable application function with validated inputs and a useful result. |

| | |
|---|---|
| MCP integration | One working client/server connection through which an agent discovers and invokes a tool. |
| Skill | One separately stored instruction package that an agent discovers and loads when relevant. |

External systems such as the MCP server can be simulated and provide mock company data. For example, a native tool could return a simulated service status, and an MCP server could provide fictional IT and HR knowledge. You may choose different examples.
Tools should have clear descriptions, validated inputs and understandable result contracts.
For this challenge, a skill is a reusable set of instructions loaded into an agent's context when needed. Agents may initially receive only its name and description; the full instructions should be loaded on demand. Permanently inserting the skill into a system prompt does not meet this requirement. The skill format and loading mechanism are your choice. Demonstrate at least one execution in which the skill is loaded and its instructions are applied.

# 3. Users, roles and permissions

Provide two predefined users or roles with meaningfully different permissions. A simple user switcher (for example a dropdown or a request header) is enough; a login screen and production identity integration are out of scope.
Define your roles and their capabilities in the README. Your permission model must distinguish between:

- What the current user may access or perform.
- What each agent may access or perform.

Implement at least:

- One capability available to one user role but not to the other.
- One capability available to one agent but not to the other.

Expose only capabilities permitted for both the current user and the executing agent. Delegation must preserve the user's permissions. Agents should be transparent about unavailable data, tools or capabilities and should not offer them.

# 4. Web interface, conversation state and trace

Build a simple React chat UI connected to the orchestrator that:

- Sends messages and shows responses, keeping context within an active session.
- Shows a basic "work in progress" indicator.
- Surfaces clear errors and capability limits.

Streaming, polished design and separate specialist chats are optional. Refresh and restart recovery is optional; document your implementation's limits.
Provide a simple structured execution trace (no private reasoning, no credentials). It does not need to be shown in the UI; logs, an API response or test output are fine. It should cover:

- Specialist delegation and completion
- Skill loading
- Tool calls and outcomes
- Permission decisions

Include timestamps so that concurrent execution is verifiable.

# 5. Technology choices

Use strict TypeScript, React, Cloudflare for the backend, an LLM API with tool calling, MCP for one integration, and automated tests. Choose the remaining libraries and Cloudflare services yourself, prefer established components, and explain key trade-offs. Local execution is sufficient.
Any LLM provider with tool calling is fine, including Workers AI. Document the model you used and the configuration we need to run it.

# 6. Demonstration scenarios

Demonstrate each scenario by running the real application with a real LLM, not a mocked one. Use realistic prompts, and fixture data only for external systems (e.g., the fictional HR policy served by the MCP server). The MCP server must offer a way to simulate an error or timeout for scenario 7. Document each scenario as an example prompt with the observed trace. Response wording will vary, so judge the trace (delegation, skills, tools, permissions), not the text.

| # | Scenario | Expected behaviour |
|---|---|---|
| 1 | IT support question | IT delegation, skill loading, native tool use |
| 2 | Fictional HR policy question | HR delegation, MCP-backed retrieval |
| 3 | Independent IT and HR questions in one request | Concurrent execution, combined response |
| 4 | IT recommendation depending on an HR policy result | Sequential execution with explicit result passing (for example, HR identifies an equipment profile and IT uses it for setup guidance) |
| 5 | User requests a capability their role lacks (for example, a password reset) | Capability not exposed to any agent; the orchestrator explains it cannot do it |
| 6 | Specialist is delegated a task needing a capability it lacks (for example, HR asked to reset a password) | Capability not exposed to that specialist; the specialist reports the limitation back to the orchestrator |
| 7 | Tool or integration failure (MCP server in failure mode) | The orchestrator explains the limitation without claiming success |

# 7. Deliverables

Submit runnable source code and a README of about one page (setup instructions may be extra) that covers:

- Setup instructions, required configuration, test users and steps to run the demonstration scenarios.
- Your roles and their capabilities.
- A short architecture overview: agent interactions, tool and skill integration, permission model, state management and significant technical choices.
- How you used AI development tools and how you verified their output.
- Approximate time spent, known limitations and next priorities.

Stop after 8 hours and document unfinished work. A clearly explained partial implementation is acceptable. If you run short on time, prioritise in this order: the permission model and

orchestration, then MCP and the skill, then the UI and trace.